Коллоквиум по C++ ПИ 1 курс весна

github.com/int28t/hse-se-lecture-notes

Содержание

1 Вокруг языка C++ **6**

1.1 Какие стандарты языка существуют? Каков цикл выпуска новых стандартов? 6

1.2 Что такое The Zero Overhead Principle? Привести пример появления этого принципа в коде. 6

1.3 Какие основные директивы языка CMake/файлов проекта? Для чего нужна директива `add_executable`? Каковы её параметры? 7

1.4 Что такое интерфейс и что такое реализация? Что из них можно отнести к идеальному, а что — к материальному? 8

1.5 Привести конкретные примеры разделения на интерфейс и реализацию в языке C++ (модули, типы и т.д.). 8

1.6 Для чего используются заголовочные файлы в C++? Что в них обычно помещают? Что с ними обычно делают и что с ними обычно делать нельзя? Какие расширения встречаются у заголовочных файлов? Какая связь между заголовочными файлами и принципом интерфейса-реализации? 9

1.7 Для чего используются модули трансляции в C++? Что в них обычно помещают? Что с ними обычно делают и что с ними обычно делать нельзя? Какие расширения встречаются у модулей трансляции? Какая связь между модулями трансляции и принципом интерфейса-реализации? 9

2 Основы C++ **10**

2.1 Каково назначение препроцессора языка C(++)? С какими сущностями он манипулирует? Имеет ли для него принципиальное значение синтаксис и семантика языка C++? 10

2.2 На примере директивы `assert` покажите, как препроцессор меняет код программы для конфигураций сборки debug и release. 11

2.3 Что содержит и что не содержит листинг (результат работы препроцессора)? 11

2.4 Приведите пример конфигурирования текста исходного кода программы условными директивами п. для... (например, включения фрагмента кода, релевантного определенной ОС). 12

2.5 Как включить в модуль системный заголовочный файл? —//— заголовочный файл из проекта? 12

2.6 В чем заключается проблема повторного косвенного включения заголовочного ф. Привести пример. 12

2.7 Что такое header guard и какую проблему он призван предотвратить? Привести конкретный пример. 13

2.8 Можно ли использовать заголовочный ф. без h.g. и если да, то какие ограничения это накладывает? 13

2.9 Что обычно кладут в заголовочный файл — интерфейсную часть некоторого модуля foo? А что обычно не кладут — и почему? 14

2.10 Чем в C++ описание чего-л. принципиально отличается от определения чего-л.? Как можно наблюдать эффект описания чего-л. и эффект определения чего-л.? . . . 14

3 Базовые конструкции C++ **15**

3.1 Алфавит языка C++. Простая программа C++, структура, основные действующие лица . . . 15
3.2 Ключевые слова. Основные ключевые слова . . . 15
3.3 Идентификаторы, правило составления идентификаторов. Влияние кодстайла на п.с.и. . . . 16
3.4 Что такое инструкция? Какой признак в программе явно указывается на появление инструкции? Когда это не так? . . . 16
3.5 Что такое инструкция описания объекта? Приведите пример. . . . 16
3.6 Что такое инструкция описания функции? Приведите пример. . . . 16
3.7 Что такое инструкция определения и описания объекта? Приведите пример. 17
3.8 Возможно ли существование инструкции только описания без определения и наоборот? Приведите примеры. . . . 17
3.9 Каким образом в C++ производится инициализация объектов и когда? Приведите различные подходы для инициализации, что между ними общего и какие есть различия. . . . 17
3.10 Что такое блок инструкций? Можно ли использовать блок без каких-либо заголовочных инструкций перед ним? . . . 18
3.11 Что такое область видимости имени (scope)? Какой scope задаётся блоком инструкций? . . . 18
3.12 Что такое функция с точки зрения языка C++? . . . 18
3.13 Объекты и типы. Характеристики объектов и характеристики типов. . . . 18
3.14 Представление объектов в оперативной памяти (footprint/dump), влияние на представления размера объекта и способа его кодирования. Оператор sizeof(). 19
3.15 Операционная семантика объекта определенного типа (операции). Абстрактный тип данных. . . . 19
3.16 Классификация типов в C++ (трехтаксонная) и по cppreference (двухтаксонная). . . . 19
3.17 Целочисленные типы данных: размеры, знаковость, диапазоны значений. Псевдонимы фиксированного размера, кодирование, Little Endian. . . . 20
3.18 Диапазоны значений ц/ч. типов: `<climits>` и `<limits>`. . . . 21
3.19 Типы чисел с плавающей точкой float и double. Размер и структура. . . . 21
3.20 Инициализация числовых объектов. Литералы, префиксы/суффиксы. Narrowing. 21
3.21 POD-типы, особенность. . . . 22
3.22 Выражения и операторы. Классификация операторов, арность, приоритетность. Инструкция-выражение. Различие между инициализацией и присваиванием. . . . 22
3.23 Логические выражения и инструкции управления потоком исполнения программы (CF). . . . 22
3.24 Может ли функция иметь более одной точки выхода с помощью инструкции return? Может ли процедура (void-функция) иметь в теле инструкцию return? 23

4 Потоки и строки STL **23**

4.1 Что подразумевают под моделью ввода-вывода консольного приложения (CLI-application)? Какие основные компоненты ввода-вывода задействуются для него? . . . 23

4.2 Что такое стандартный поток ввода и стандартный поток вывода? Какими объектами они представлены? Какая область видимости этих объектов? . . 23
4.3 Какой заголовочный файл необходимо подключить, чтобы получить доступ к стандартным потокам ввода/вывода? Где они описаны и где определены? 23
4.4 Какие типы данных у стандартных потоков ввода и вывода? Какая иерархия наследования для потоковых типов STL? 24
4.5 Какие операции используются для ввода и вывода? Можно ли использовать оператор ввода для потока вывода и наоборот? Как они работают с пробелами? Что возвращают? 24
4.6 Какие функции ввода и вывода можно использовать наравне с операторами? Как прочитать целиком строку, содержащую пробелы? 24
4.7 Что такое поток? Почему поток называют абстракцией? 24
4.8 Что такое C-строка? Чем она отличается от обычного массива символов? Что такое null terminator? 25
4.9 Что происходит со строковым литералом в C/C++? 25
4.10 Что такое `std::string`? Как он связан с `std::basic_string<char>`? 25
4.11 Какие базовые операции есть у `std::string`? 25
4.12 Как соотносятся размер объекта `std::string` и длина хранимой в нём строки? Что такое оптимизация коротких строк? 26
4.13 Что такое `std::stringstream` и зачем он нужен? 26

5 Функциональная декомпозиция, ссылки и константность 26
5.1 Как решить большую задачу так, чтобы полученное решение было корректным, проверяемым, воспроизводимым, переиспользуемым, читаемым? 26
5.2 Что такое функция с точки зрения языка C++? Как определить функцию/-процедуру? Какие основные компоненты функции C++? 27
5.3 Что такое неполный тип void? Можно ли породить объект такого типа? а типа void* — и почему? 27
5.4 Что такое свободная функция (non-member function) и что такое функция-член (member function)? Привести примеры. 27
5.5 Является ли функция `std::getline` свободной или функцией-членом? . . . 27
5.6 Является ли статическая функция (помечена ключевым словом static) `bar()`, объявленная в классе Foo, свободной или функцией-членом? Как вызвать такую функцию? 28
5.7 Что такое side-effect при вызове свободных функций и почему его наличие считается нежелательным? Как с помощью static можно внести в функцию некоторое состояние? 28
5.8 Что такое методы для классов и структур? Приведите пример метода для класса `std::string`. 28
5.9 Что такое список формальных параметров функции? Можно ли составить список ф.п., состоящий только из типов? 29
5.10 Что такое сигнатура функции? Для чего необходимо рассматривать сигнатуру отдельно от заголовка функции? 29
5.11 Чем различаются заголовок функции от прототипа функции? Что из них относится к интерфейсной части функции? 29
5.12 Параметры функции: передача по значению и по ссылке. Когда, зачем и почему? 29
5.13 Что такое константная ссылка? 29
5.14 Идиоматический приём: константная ссылка в range-based for. 30
5.15 «Большие» и «малые» объекты: правило трёх указателей. 30

5.16 С какой целью функция, получающая параметр по ссылке, может возвращать тот же параметр тоже по ссылке (pass-through)? . . . 30

6 Стандартная библиотека: основные концепции и линейные контейнеры 31

6.1 Что такое стандартная библиотека C++ и зачем она нужна? . . . 31
6.2 Что такое контейнер STL? Какие большие группы контейнеров выделяют? . 31
6.3 Какие контейнеры относятся к линейным? . . . 31
6.4 Что такое `std::vector`? В чём его главные свойства? . . . 31
6.5 Чем `std::vector` отличается от C-массива и от `std::array`? . . . 32
6.6 Что такое size и capacity у `std::vector`? . . . 32
6.7 Для чего нужен `reserve()`? . . . 32
6.8 Что такое матрица в контексте STL-решений курса? . . . 32
6.9 Что такое алиас типа? Чем `using` отличается от `typedef`? . . . 32
6.10 Приведите пример алиасов для ряда и зрительного зала. . . . 33

7 Стандартная библиотека: итераторы 33

7.1 Что такое итератор? . . . 33
7.2 Что означают `begin()` и `end()`? Почему интервал записывают как `[begin, end)`? . . . 33
7.3 Можно ли сделать копию итератора `c.begin()`? Можно ли эту копию разыменовать? Можно ли эту копию двигать вперёд? . . . 33
7.4 Можно ли сделать копию итератора `c.end()`? Можно ли эту копию разыменовать? Можно ли эту копию двигать назад? . . . 33
7.5 Какой вариант сравнения итераторов при переборе коллекции следует предпочесть: `it != end()` или `it < end()`? Почему? . . . 34
7.6 Какие категории итераторов выделяли до стандарта C++20? Чем они различаются? Как категории соотносятся друг с другом? . . . 34
7.7 При разработке нового алгоритма какую категорию итераторов следует требовать? . . . 34
7.8 Что происходит с итератором после удаления элемента из коллекции? Как следует поступить с этим итератором после удаления элемента? . . . 34

8 Косвенная адресация 35

8.1 Что такое косвенная адресация? Какие основные варианты косвенности разбирались в курсе? . . . 35
8.2 В чём общая идея указателя, ссылки и итератора как средств косвенности? 35
8.3 Чем указатель отличается от ссылки? . . . 35
8.4 Что значит левоконстантный, правоконстантный и дваждыконстантный указатель? . . . 36
8.5 Что можно сказать о константной и неконстантной ссылке? . . . 36
8.6 Что такое `const_iterator`, `const iterator` и `const const_iterator`? . . . 36

9 Класс std::string_view 36

9.1 Что такое `std::string_view`? . . . 36
9.2 Почему `std::string_view` называют невладеющим типом? . . . 36
9.3 Почему `std::string_view` удобно передавать по значению? . . . 36
9.4 Можно ли модифицировать строку через `std::string_view`? . . . 37
9.5 Гарантирует ли `std::string_view` наличие `'\0'` в конце? . . . 37
9.6 В каком случае `std::string_view` особенно хорош? . . . 37
9.7 Как `std::string_view` связан с идеей косвенной адресации? . . . 37

10 Ассоциативные контейнеры std::set и std::map **37**
10.1 Что такое `std::set`? . . . 37
10.2 Что такое `std::map`? . . . 37
10.3 Что является элементом `std::map`? . . . 37
10.4 Какой тип обычно используют для представления пары объектов? . . . 37
10.5 Какая типичная внутренняя реализация у `std::set` и `std::map`? . . . 38
10.6 Какова асимптотика основных операций у `std::set` и `std::map`? . . . 38
10.7 Почему при поиске и вставке в `set`/`map` используется сравнение? . . . 38
10.8 Почему для `std::set` обычно достаточно `operator<`? . . . 38
10.9 Можно ли хранить в `set` дубликаты? . . . 38
10.10 Можно ли хранить в `map` несколько одинаковых ключей? . . . 38

11 Ассоциативные контейнеры, компараторы, сравнение и сортировка **38**
11.1 Каково назначение параметра `Compare` в шаблонах `std::set<Key, Compare>` и `std::map<Key, Value, Compare>`? . . . 38
11.2 Что такое компаратор? . . . 39
11.3 Что сравнивает компаратор у `std::set` и у `std::map`? . . . 39
11.4 Что такое `std::less`? . . . 39
11.5 Можно ли задать свой компаратор? . . . 39
11.6 Что такое свободная функция в роли компаратора? . . . 39
11.7 Почему компаратор должен задавать устойчивое и разумное правило порядка? 39
11.8 Какая типичная внутренняя реализация у `std::map` и какова асимптотика операций? . . . 40
11.9 Можно ли использовать один и тот же компаратор в разных контейнерах? . 40

1 Вокруг языка C++

1.1 Какие стандарты языка существуют? Каков цикл выпуска новых стандартов?

Язык C++ стандартизуется. У него есть последовательность стандартов, в которых фиксируются возможности языка и стандартной библиотеки.

Основные стандарты C++

Обычно выделяют следующие крупные ревизии языка:

- **C++98**;
- **C++03**;
- **C++11**;
- **C++14**;
- **C++17**;
- **C++20**;
- **C++23**.

Для курса важно помнить отдельно:

- **базовый стандарт курса — C++17**;
- **отдельные элементы C++20** тоже используются и обсуждаются.

До окончательного принятия нового стандарта существует его рабочий проект — *draft*.

Примечание

В современный период новые стандарты C++ выходят примерно **раз в три года**. На защите полезно разделять:

- общую историю стандартов;
- конкретную договорённость курса: C++17 + отдельные элементы C++20.

1.2 Что такое The Zero Overhead Principle? Привести пример появления этого принципа в коде.

The Zero Overhead Principle

Принцип нулевых накладных расходов означает: если программист не использует некоторую абстракцию, программа не должна за неё “платить”; если использует, цена этой абстракции должна быть минимальной.

Это одна из ключевых идей C++: язык даёт высокоуровневые средства, но старается не превращать их в обязательные лишние runtime-расходы.

Пример

Простой пример — вызов небольшой функции или шаблонной функции:

```
template <class T>
T myMax(const T& a, const T& b)
{
    return (a < b) ? b : a;
}
```

С точки зрения программиста это удобная абстракция. После инстанцирования и оптимизации компилятор обычно получает код, который по стоимости близок к ручной реализации для конкретного типа.

Примечание

На этом вопросе не нужно уходить в слишком глубокую теорию оптимизаций. Важно правильно объяснить сам принцип.

1.3 Какие основные директивы языка CMake/файлов проекта? Для чего нужна директива `add_executable`? Каковы её параметры?

В курсе все проекты создаются на базе CMake.

Типичные команды CMake

В простом проекте чаще всего встречаются:

- `cmake_minimum_required(...)` — минимальная версия CMake;
- `project(...)` — имя проекта и язык;
- `add_executable(...)` — создание исполняемого таргета;
- `add_library(...)` — создание библиотечного таргета.

`add_executable`

Команда `add_executable` создаёт **target**, результатом сборки которого является исполняемый файл.

В простейшем случае у неё два главных смысловых параметра:

- имя таргета;
- список исходных файлов.

Пример

```
cmake_minimum_required(VERSION 3.5)
project(WorkshopX CXX)

add_executable(ex1 ex1/main.cpp)
```

Примечание

На защите лучше использовать слово **target**, потому что именно этот термин используется в лекциях.

1.4 Что такое интерфейс и что такое реализация? Что из них можно отнести к идеальному, а что — к материальному?

Интерфейс

Интерфейс объявляет что-то без задания конкретного способа реализации и тем самым задаёт **контракт** между частями программы.

Реализация

Реализация претворяет этот контракт в жизнь одним из возможных способов.

Интерфейс отвечает на вопрос *что можно делать*, а реализация — *как это устроено*. Одному и тому же интерфейсу может соответствовать несколько разных реализаций. С идейной точки зрения:

- **интерфейс** относится к более **идеальной** стороне;
- **реализация** относится к более **материальной** стороне.

1.5 Привести конкретные примеры разделения на интерфейс и реализацию в языке C++ (модули, типы и т.д.).

Типичные примеры в C++:

- **Прототип функции** — интерфейс функции.
- **Тело функции** — реализация.
- **Создание нового типа** — часть интерфейсного описания.
- **Создание объекта конкретного типа** — уже реализационная часть.
- **Интерфейсный класс** — класс, состоящий только из чисто виртуальных функций.
- **Наследование от интерфейсного класса и override виртуальных функций** — реализация интерфейса.

Пример

```
double power(double x, unsigned int p);   // интерфейс

double power(double x, unsigned int p)
{
    double res = x;
    for (unsigned int i = 1; i < p; ++i)
        res = res * x;
    return res;
}
```

1.6 Для чего используются заголовочные файлы в C++? Что в них обычно помещают? Что с ними обычно делают и что с ними обычно делать нельзя? Какие расширения встречаются у заголовочных файлов? Какая связь между заголовочными файлами и принципом интерфейса-реализации?

Заголовочный файл

Заголовочный файл — это, как правило, **интерфейсная часть** некоторого модуля.

Обычно в заголовочные файлы кладут:

- прототипы функций;
- описания пользовательских типов;
- интерфейсные классы;
- в ряде случаев — шаблоны целиком.

Типичные расширения:

- `.h`;
- `.hpp`;
- `.hxx`.

Что с ними обычно делают:

- подключают через `#include` в `.cpp`;
- через них распространяют интерфейс между модулями.

Что обычно делать не стоит:

- помещать туда обычные определения объектов и функций, если заголовок будет включаться более чем в один модуль трансляции;
- класть туда детали реализации, не относящиеся к интерфейсу.

Примечание

Заголовочный файл в обычной организации проекта соответствует именно **интерфейсу**.

1.7 Для чего используются модули трансляции в C++? Что в них обычно помещают? Что с ними обычно делают и что с ними обычно делать нельзя? Какие расширения встречаются у модулей трансляции? Какая связь между модулями трансляции и принципом интерфейса-реализации?

Модуль трансляции

Модуль трансляции (*translation unit*) — это исходный файл C++, который после работы препроцессора компилируется как единое целое.

В рамках обычной организации проекта это, как правило, `.cpp`-файл.
Обычно в модуле трансляции помещают:

- реализации функций;
- определения объектов;
- детали реализации, скрытые от внешнего кода.

Что с ними делают:

- препроцессируют;
- компилируют;
- затем линкуют результат в программу.

Типичное расширение:

- `.cpp`.

Важно различать:

- `.cpp` — отдельный модуль трансляции;
- `.h`, `.hpp`, `.hxx` — заголовок, а не отдельный компилируемый модуль.

Примечание

Если заголовок — это обычно интерфейс, то модуль трансляции — это обычно **реализация**.

2 Основы C++

2.1 Каково назначение препроцессора языка C(++)? С какими сущностями он манипулирует? Имеет ли для него принципиальное значение синтаксис и семантика языка C++?

Препроцессор

Препроцессор — этап обработки текста программы *до* компиляции.

Его назначение:

- подключать заголовочные файлы;
- разворачивать макросы;
- выполнять условное включение или исключение фрагментов текста;
- конфигурировать текст программы под разные режимы сборки, ОС и платформы.

Препроцессор работает прежде всего с **текстом** исходника. Он манипулирует:

- директивами;
- токенами;
- текстовыми фрагментами.

Для него не имеет принципиального значения полная семантика C++: он не понимает типы, классы, перегрузку и т.д. Его работа носит текстовый характер.

Примечание

Полезная формулировка: препроцессор подготавливает текст исходного кода под конкретные условия билда.

2.2 На примере директивы `assert` покажите, как препроцессор меняет код программы для конфигураций сборки debug и release.

`assert` подключается через:

```
#include <cassert>
```

Пример

```
#include <cassert>

int f(int x)
{
    assert(x != 0);
    return 100 / x;
}
```

Если макрос `NDEBUG` **не определён**, проверка остаётся в программе и при ложном условии завершает выполнение с диагностикой.

Если `NDEBUG` **определён**, макрос `assert(...)` разворачивается в пустую безопасную конструкцию вида `((void)0)` и фактически исчезает из runtime-поведения программы.

Это пример того, как один и тот же текст исходника конфигурируется по-разному для debug и release.

2.3 Что содержит и что не содержит листинг (результат работы препроцессора)?

Листинг

Листинг — это текст программы после работы препроцессора.

Листинг обычно содержит:

- результаты `#include`;
- раскрытые макросы;

- уже выбранные ветки условной компиляции;
- служебные строки, помогающие соотносить результат с исходными файлами.

Листинг обычно не содержит в исходном виде:

- самих директив `#include`, `#define` как управляющих конструкций;
- комментариев как самостоятельной сущности;
- результата семантического анализа программы компилятором.

2.4 Приведите пример конфигурирования текста исходного кода программы условными директивами п. для... (например, включения фрагмента кода, релевантного определенной ОС).

Пример

```
#ifdef _WIN32
    // код только для Windows
#elif defined(__linux__)
    // код только для Linux
#else
    // код для остальных систем
#endif
```

Другой частый пример — условное включение отладочного кода:

```
#ifdef DEBUG
    std::cerr << "debug mode\n";
#endif
```

2.5 Как включить в модуль системный заголовочный файл? —//— заголовочный файл из проекта?

Системный заголовок подключают так:

```
#include <iostream>
```

Проектный заголовок подключают так:

```
#include "foo.h"
```

Разница в синтаксисе отражает различный способ поиска файла:

- `<...>` — для системных путей;
- `"..."` — сначала локальный поиск, потом системные пути.

2.6 В чем заключается проблема повторного косвенного включения заголовочного ф. Привести пример.

Проблема возникает, когда один и тот же заголовочный файл включается в один и тот же модуль трансляции несколько раз через цепочку других заголовков.

Пример

```
// foo.h
#ifndef FOO_H_
#define FOO_H_

double power(double x, unsigned int p);
int GLOBAL_I;

#endif

// bar.h
#ifndef BAR_H_
#define BAR_H_

#include "foo.h"
double power2(double x, double p);

#endif

// main.cpp
#include "foo.h"
#include "bar.h"
```

Даже если повторное текстовое включение самого файла уже предотвращено, в заголовке всё равно может остаться “мина” — например, `int GLOBAL_I;`. Такой заголовок можно включить только в один модуль трансляции, иначе будут проблемы на этапе линковки.

2.7 Что такое header guard и какую проблему он призван предотвратить? Привести конкретный пример.

Header guard

Header guard — это защита заголовочного файла от повторного включения.

Классическая реализация:

```
#ifndef PROJECT_MATH_VECTOR_H
#define PROJECT_MATH_VECTOR_H

// содержимое заголовка

#endif
```

Header guard предотвращает повторное текстовое включение содержимого одного и того же заголовка в один модуль трансляции.

2.8 Можно ли использовать заголовочный ф. без h.g. и если да, то какие ограничения это накладывает?

Теоретически можно, но только при очень жёстком ограничении: такой заголовок должен попасть в каждый модуль трансляции не более одного раза.

На практике это неудобно и небезопасно. Поэтому нормальная практика — использовать либо классический header guard, либо `#pragma once`.

2.9 Что обычно кладут в заголовочный файл — интерфейсную часть некоторого модуля foo? А что обычно не кладут — и почему?

Обычно в заголовочный файл кладут:

- прототипы функций;
- описания типов;
- объявления объектов через `extern`, если это действительно нужно;
- шаблоны.

Обычно не кладут:

- определения обычных объектов;
- определения обычных нешаблонных функций;
- детали реализации, не относящиеся к интерфейсу.

Причина — при включении такого заголовка в несколько модулей трансляции можно получить ошибки компиляции или линковки.

2.10 Чем в C++ описание чего-л. принципиально отличается от определения чего-л.? Как можно наблюдать эффект описания чего-л. и эффект определения чего-л.?

Описание

Описание (*declaration*) создаёт для компилятора знание о том, что в программе есть некоторая сущность и как с ней можно работать.

Определение

Определение (*definition*) создаёт реальный объект в памяти, либо даёт тело функции, то есть реальную сущность в программе.

Простейшая формулировка:

- **описать** — значит оповестить компилятор;
- **определить** — значит реально создать сущность.

Пример

```
extern int x;   // только описание
int x = 5;      // определение и одновременно описание
```

Для функций:

```
int sum(int, int);        // описание
int sum(int a, int b)     // определение
{
    return a + b;
}
```

Эффект описания наблюдается в том, что компилятор “знает имя” сущности. Эффект определения наблюдается в том, что сущность начинает реально существовать в программе.

3 Базовые конструкции C++

3.1 Алфавит языка C++. Простая программа C++, структура, основные действующие лица

Алфавит C++ включает:

- латинские буквы;
- цифры;
- знак подчёркивания;
- специальные символы операторов и разделителей;
- символы препроцессора;
- символы пробельных разделителей.

Минимальная программа:

```
int main()
{
    return 0;
}
```

Основные “действующие лица” при построении программы:

- препроцессор;
- компилятор;
- линкер;
- runtime-среда.

3.2 Ключевые слова. Основные ключевые слова

Ключевое слово — зарезервированное слово языка, которое нельзя использовать как обычный идентификатор.

Примеры:

- типы и объявления: `int`, `double`, `char`, `void`, `class`, `struct`, `enum`;
- управление потоком: `if`, `else`, `switch`, `for`, `while`, `do`, `break`, `continue`, `return`;
- спецификаторы: `const`, `static`, `extern`, `inline`, `virtual`, `namespace`, `auto`.

3.3 Идентификаторы, правило составления идентификаторов. Влияние кодстайла на п.с.и.

Идентификатор:

- состоит из букв, цифр и символа `_`;
- не может начинаться с цифры;
- чувствителен к регистру;
- не должен совпадать с ключевым словом.

Кодстайл накладывает дополнительные соглашения:

- типы часто именуют в `PascalCase`;
- функции и переменные — в `camelCase`;
- макросы — в `UPPER_CASE`;
- глобальные константы тоже часто выделяют отдельным стилем.

3.4 Что такое инструкция? Какой признак в программе явно указывается на появление инструкции? Когда это не так?

Инструкция

Инструкция (*statement*) — законченная синтаксическая конструкция языка.

Часто признаком инструкции служит:

- точка с запятой `;`;
- либо фигурные скобки для составной инструкции.

Но это не всегда так:

- `if`, `switch`, циклы не обязательно заканчиваются `;` как целое;
- блок `{...}` сам является составной инструкцией.

3.5 Что такое инструкция описания объекта? Приведите пример.

Это инструкция, которая сообщает компилятору о существовании объекта и его типе, но не обязательно создаёт его.

Пример

```
extern int counter;
```

3.6 Что такое инструкция описания функции? Приведите пример.

Это инструкция, которая задаёт интерфейс функции без тела.

Пример

```
double power(double x, unsigned int p);
```

Такой заголовок функции вместе с `;` называется прототипом.

3.7 Что такое инструкция определения и описания объекта? Приведите пример.

Это инструкция, которая одновременно и описывает объект для компилятора, и реально создаёт его.

Пример

```
int a = 42;
```

3.8 Возможно ли существование инструкции только описания без определения и наоборот? Приведите примеры.

Только описание без определения — возможно:

```
extern int x;
int sum(int, int);
```

Определение без отдельной инструкции описания тоже возможно, потому что определение обычно одновременно является и описанием:

```
int x = 5;
int sum(int a, int b) { return a + b; }
```

3.9 Каким образом в C++ производится инициализация объектов и когда? Приведите различные подходы для инициализации, что между ними общего и какие есть различия.

Инициализация происходит **в момент создания объекта**.
Основные формы:

- `T x = value;` — copy-initialization;
- `T x(value);` — direct-initialization;
- `T x{value};` — list-initialization;
- `T x = {value};` — copy-list-initialization.

Общее: все они задают начальное значение создаваемому объекту.
Различия:

- `{}` лучше обнаруживает narrowing;
- `=` в объявлении выглядит как присваивание, но здесь это именно инициализация;
- `()` и `{}` различаются правилами выбора конструкторов и поведением при сужении.

3.10 Что такое блок инструкций? Можно ли использовать блок без каких-либо заголовочных инструкций перед ним?

Блок инструкций — это последовательность инструкций в фигурных скобках:

```
{
    int x = 1;
    int y = x + 2;
}
```

Да, блок можно использовать и без заголовка `if`/`for`/функции. Это создаёт отдельную локальную область видимости.

3.11 Что такое область видимости имени (scope)? Какой scope задаётся блоком инструкций?

Scope — область программы, внутри которой имя видно и доступно без дополнительной квалификации.

Блок инструкций задаёт **локальную область видимости**.

3.12 Что такое функция с точки зрения языка C++?

Функция — именованный блок инструкций с заданным интерфейсом:

- имя;
- возвращаемый тип;
- список параметров;
- тело.

Функция задаёт контракт вызова и реализует некоторое действие или вычисление.

3.13 Объекты и типы. Характеристики объектов и характеристики типов.

Объект — конкретная область памяти с данными.

Тип — описание множества значений и множества операций над ними.

Характеристики объекта:

- тип;
- значение;
- адрес;
- размер;
- время жизни;
- имя, если оно есть.

Характеристики типа:

- множество значений;
- набор операций;
- размер;
- способ кодирования;
- структура представления.

3.14 Представление объектов в оперативной памяти (footprint/dump), влияние на представления размера объекта и способа его кодирования. Оператор sizeof().

Объект реально располагается в памяти и имеет некоторый *footprint* — занимаемый объём и структуру байтов.

На представление влияют:

- размер типа;
- способ кодирования данных;
- выравнивание;
- внутренняя структура типа.

Оператор `sizeof` возвращает размер типа или объекта в байтах:

```
sizeof(int)
sizeof x
```

3.15 Операционная семантика объекта определенного типа (операции). Абстрактный тип данных.

Операционная семантика типа — это набор допустимых операций над объектами данного типа и смысл этих операций.

Например, для `int` это арифметика, сравнение, присваивание, инкремент и т.д.

Абстрактный тип данных — это тип, для которого важнее интерфейс операций, чем конкретная реализация хранения.

3.16 Классификация типов в C++ (трехтаксонная) и по cppreference (двухтаксонная).

Трёхтаксонная классификация:

- фундаментальные;
- производные;
- пользовательские.

Примеры:

- фундаментальные: `int`, `double`, `char`, `bool`, `void`;

- производные: указатели, ссылки, массивы, функции;
- пользовательские: `class`, `struct`, `enum`, `union`.

По cppreference часто используют более грубое деление:

- fundamental;
- compound.

3.17 Целочисленные типы данных: размеры, знаковость, диапазоны значений. Псевдонимы фиксированного размера, кодирование, Little Endian.

К целочисленным типам относятся:

- `char`;
- `short`;
- `int`;
- `long`;
- `long long`;

и их unsigned-варианты.

Размеры зависят от платформы, но для курса важно помнить:

- `char` часто 1 байт;
- `int` часто 4 байта.

Псевдонимы фиксированного размера из `<cstdint>`:

- `std::int8_t`, `std::uint8_t`;
- `std::int16_t`, `std::uint16_t`;
- `std::int32_t`, `std::uint32_t`;
- `std::int64_t`, `std::uint64_t`.

Little Endian означает, что младший байт многобайтового числа расположен по меньшему адресу.

Пример

Если `char c = 'A';` и `int i = 'A';`, а код символа `'A'` равен `65_10 = 41_16`, то в памяти:

- `c`: `41`;
- `i`: `41 00 00 00`.

3.18 Диапазоны значений ц/ч. типов: <climits> и <limits>.

<climits> даёт макросы:

- INT_MIN;
- INT_MAX;
- и аналогичные для других целых типов.

<limits> даёт шаблон:

```
std::numeric_limits<T>
```

через который можно получить свойства типа более общим и “плюсовым” способом.

3.19 Типы чисел с плавающей точкой float и double. Размер и структура.

Основные типы:

- float;
- double;
- long double.

Обычно:

- float занимает 4 байта;
- double — 8 байт.

Внутренняя структура обычно включает:

- знак;
- порядок;
- мантиссу.

3.20 Инициализация числовых объектов. Литералы, префиксы/-суффиксы. Narrowing.

Примеры суффиксов:

- 1u;
- 1l;
- 1ll;
- 1.0f;
- 1.0l.

Narrowing — сужение типа с возможной потерей информации.
Пример:

```
int x = 3.14;   // может пройти
int y{3.14};    // ошибка: narrowing
```

Именно поэтому инициализация через {} считается более безопасной.

3.21 POD-типы, особенность.

POD — историческое обозначение “простых” типов данных без сложной пользовательской логики.

Для учебного уровня важно помнить:

- такие типы имеют простое и предсказуемое представление в памяти;
- для них проще рассуждать о побайтном копировании и footprint.

3.22 Выражения и операторы. Классификация операторов, арность, приоритетность. Инструкция-выражение. Различие между инициализацией и присваиванием.

Выражение — конструкция, которая вычисляется.

Оператор — обозначение операции. По арности операторы бывают:

- унарные;
- бинарные;
- тернарные.

Приоритет задаёт порядок вычисления.

Инструкция-выражение — выражение, завершённое `;`:

```
x = y + 1;
f();
```

Важно различать:

- **инициализация** — в момент создания объекта;
- **присваивание** — после того, как объект уже создан.

3.23 Логические выражения и инструкции управления потоком исполнения программы (CF).

Логические операции:

- `&&`;
- `||`;
- `!`;

Операторы сравнения:

- `==`, `!=`;
- `<`, `<=`, `>`, `>=`.

Основные CF-конструкции:

- `if`, `if..else`;
- `if` с инициализацией;

- `switch`, `switch` с инициализацией;
- тернарный оператор;
- `while`, `for`, `range-based for`, `do..while`;
- `break`, `continue`, `return`.

3.24 Может ли функция иметь более одной точки выхода с помощью инструкции return? Может ли процедура (void-функция) иметь в теле инструкцию return?

Да, функция может иметь несколько инструкций `return`. Это допустимо, хотя в разных стилях программирования к этому относятся по-разному.

Да, `void`-функция тоже может содержать `return;`. В таком случае это просто досрочное завершение функции без возвращаемого значения.

4 Потоки и строки STL

4.1 Что подразумевают под моделью ввода-вывода консольного приложения (CLI-application)? Какие основные компоненты ввода-вывода задействуются для него?

CLI-приложение взаимодействует с пользователем через текстовый ввод и вывод в консоли.

Главные компоненты:

- стандартный ввод;
- стандартный вывод;
- при необходимости — файловый ввод и вывод.

Полезная модель из лекции: консольное приложение можно рассматривать как “чёрный ящик”, который получает данные через поток ввода и отправляет данные в поток вывода.

4.2 Что такое стандартный поток ввода и стандартный поток вывода? Какими объектами они представлены? Какая область видимости этих объектов?

Стандартный поток ввода — `std::cin`. Стандартный поток вывода — `std::cout`.

Это глобальные объекты стандартной библиотеки из пространства имён `std`.

4.3 Какой заголовочный файл необходимо подключить, чтобы получить доступ к стандартным потокам ввода/вывода? Где они описаны и где определены?

Для работы со стандартными потоками подключают:

```
#include <iostream>
```

В заголовке они **объявлены**, а реальные объекты **определены** в стандартной библиотеке.

4.4 Какие типы данных у стандартных потоков ввода и вывода? Какая иерархия наследования для потоковых типов STL?

- `std::cin` имеет тип `std::istream`;
- `std::cout` имеет тип `std::ostream`.

Более общая иерархия включает:

- `ios_base`;
- `basic_ios`;
- `basic_istream`;
- `basic_ostream`;
- `basic_iostream`;
- файловые и строковые производные потоки.

4.5 Какие операции используются для ввода и вывода? Можно ли использовать оператор ввода для потока вывода и наоборот? Как они работают с пробелами? Что возвращают?

Для ввода используется », для вывода — «.

- `std::cin » x;` — вводит данные из потока;
- `std::cout « x;` — выводит данные в поток.

Оператор » обычно читает токены, разделённые пробелами. Оператор « выводит данные как есть.

Оба оператора возвращают **ссылку на поток**, поэтому можно строить цепочки:

```
std::cout << a << " " << b;
std::cin >> x >> y;
```

4.6 Какие функции ввода и вывода можно использовать наравне с операторами? Как прочитать целиком строку, содержащую пробелы?

Для чтения целой строки с пробелами используют:

```
std::getline(std::cin, s);
```

Это свободная функция, а не метод строки.

В отличие от », она читает строку целиком до символа конца строки.

4.7 Что такое поток? Почему поток называют абстракцией?

Поток — абстракция для ввода или вывода.

Его называют абстракцией, потому что программист работает не с конкретной клавиатурой, экраном или файлом напрямую, а с единым интерфейсом потока символов.

4.8 Что такое C-строка? Чем она отличается от обычного массива символов? Что такое null terminator?

C-строка — это массив символов, который оканчивается `'\0'`.
Обычный массив символов не обязан содержать завершающий нулевой символ.

Пример

```
char a[8] = {'j','u','s','t',' ','a','r','r'};       // не строка
char s[8] = {'c','s','t','r','i','n','g','\0'};      // C-строка
```

`null terminator` — это **число 0**, а не символ цифры `'0'`.

4.9 Что происходит со строковым литералом в C/C++?

Компилятор превращает строковый литерал в массив символов, то есть в C-строку, размер которой равен числу видимых символов плюс один символ `'\0'`.

Пример

```
char str[] = "Otter";
```

Здесь реально создаётся массив из шести символов: `O t t e r \0`.

4.10 Что такое `std::string`? Как он связан с `std::basic_string<char>`?

`std::string` — стандартный тип C++ для представления строк.
Он является псевдонимом:

```
std::string == std::basic_string<char>
```

Обычно `std::string` хранит непрерывную последовательность 8-битных символов.

4.11 Какие базовые операции есть у `std::string`?

Часто используемые операции:

- `s.size()`, `s.length()`;
- `s.clear()`;
- `s.empty()`;
- `s[i]`;
- `s + t`;
- `s.c_str()`.

`c_str()` возвращает указатель на внутренний буфер строки.

4.12 Как соотносятся размер объекта `std::string` и длина хранимой в нём строки? Что такое оптимизация коротких строк?

Размер объекта `std::string` и длина строки — это не одно и то же.
Сам объект строки содержит служебные поля, например:

- текущую длину;
- capacity;
- указатель на буфер, либо внутренний локальный буфер.

Поэтому `sizeof(std::string)` не равен длине текста внутри строки.

Оптимизация коротких строк (*SSO, short string optimization*) означает, что короткие строки могут храниться прямо внутри объекта `std::string`, без отдельной аллокации в куче.

4.13 Что такое `std::stringstream` и зачем он нужен?

`std::stringstream` — специальный поток, который сочетает свойства строки и потока.
Он полезен, когда нужно:

- разбирать строку как поток;
- собирать строку с помощью потоковых операций;
- использовать знакомый интерфейс » и « для строк.

5 Функциональная декомпозиция, ссылки и константность

5.1 Как решить большую задачу так, чтобы полученное решение было корректным, проверяемым, воспроизводимым, переиспользуемым, читаемым?

Ключевая идея — **декомпозиция**.
Большую задачу нужно:

- разбить на маленькие кусочки;
- каждый кусочек — ещё на более мелкие подзадачи;
- повторять это, пока отдельные части не станут достаточно простыми.

Хороший отдельный кусок должен быть:

- параметризуемым;
- переиспользуемым;
- проверяемым отдельно от всей большой программы.

5.2 Что такое функция с точки зрения языка C++? Как определить функцию/процедуру? Какие основные компоненты функции C++?

Функция — именованный блок инструкций с интерфейсом и телом.
Основные компоненты:

- возвращаемый тип;
- имя;
- список параметров;
- тело функции.

Процедура — это функция с возвращаемым типом `void`.

5.3 Что такое неполный тип void? Можно ли породить объект такого типа? а типа void* — и почему?

`void` — специальный неполный тип, означающий отсутствие значения.
Объект типа `void` создать нельзя.
А вот `void*` создать можно, потому что это уже указатель, и он хранит адрес, а не сам объект типа `void`.

5.4 Что такое свободная функция (non-member function) и что такое функция-член (member function)? Привести примеры.

Свободная функция не принадлежит классу:

```
int sum(int a, int b)
{
    return a + b;
}
```

Функция-член принадлежит классу:

```
struct Point
{
    double x, y;
    double norm() const { return x * x + y * y; }
};
```

5.5 Является ли функция `std::getline` свободной или функцией-членом?

`std::getline` — **свободная функция**.

5.6 Является ли статическая функция (помечена ключевым словом static) `bar()`, объявленная в классе Foo, свободной или функцией-членом? Как вызвать такую функцию?

Если функция объявлена внутри класса как `static`, то это всё равно **функция-член**, но без неявного параметра `this`.

Вызывается так:

```
Foo::bar();
```

5.7 Что такое side-effect при вызове свободных функций и почему его наличие считается нежелательным? Как с помощью static можно внести в функцию некоторое состояние?

Side-effect — это изменение состояния, выходящее за рамки "чистого" вычисления результата функции.

Он часто считается нежелательным, потому что:

- ухудшает предсказуемость;
- усложняет тестирование;
- ухудшает переиспользуемость.

Пример внутреннего состояния функции:

```
int f()
{
    static int counter = 0;
    return ++counter;
}
```

Здесь `static` сохраняет значение между вызовами и тем самым создаёт side-effect.

5.8 Что такое методы для классов и структур? Приведите пример метода для класса `std::string`.

Методы — это функции-члены классов и структур.

Примеры методов `std::string`:

- `size()`;
- `empty()`;
- `clear()`;
- `c_str()`.

5.9 Что такое список формальных параметров функции? Можно ли составить список ф.п., состоящий только из типов?

Список формальных параметров — это перечень параметров в заголовке функции.
В определении функции нужны имена параметров, если они используются в теле.
В прототипе имена можно опускать:

```
int sum(int, int);
```

То есть список может состоять только из типов, если речь идёт о прототипе.

5.10 Что такое сигнатура функции? Для чего необходимо рассматривать сигнатуру отдельно от заголовка функции?

Сигнатура функции — это часть её интерфейса, которая отличает одну перегрузку от другой.
Обычно в неё входят:

- имя функции;
- типы параметров;
- в C++ также qualifiers, если они участвуют в различении перегрузок.

Сигнатуру рассматривают отдельно, потому что именно она важна для перегрузки, а не весь текст заголовка целиком.

5.11 Чем различаются заголовок функции от прототипа функции? Что из них относится к интерфейсной части функции?

Заголовок функции — это строка с возвращаемым типом, именем и параметрами.
Если к заголовку добавить `;` и не писать тело, получится **прототип функции**.
Именно прототип относится к интерфейсной части функции в явном виде.

5.12 Параметры функции: передача по значению и по ссылке. Когда, зачем и почему?

Передача по значению означает копирование аргумента в параметр функции.
Передача по ссылке означает работу с исходным объектом без копии.
Идиоматические правила:

- большие объекты для чтения — через `const T&`;
- маленькие объекты для чтения — по значению;
- объекты для чтения и записи — через `T&`.

5.13 Что такое константная ссылка?

Константная ссылка — это ещё одно имя для объекта, через которое можно только читать, но нельзя изменять объект.

Пример

```
const std::string& s
```

Полезная интуиция из лекции: это однонаправленный “портал” только для чтения.

5.14 Идиоматический приём: константная ссылка в range-based for.

Если элементы диапазона большие, лучше итерироваться так:

```
for (const auto& el : container)
{
    ...
}
```

Это позволяет:

- не копировать каждый элемент;
- не давать возможности случайно изменить элемент;
- сохранить эффективность.

5.15 «Большие» и «малые» объекты: правило трёх указателей.

Практический критерий:

- если объект примерно не больше трёх указателей, его считают “небольшим”;
- если существенно больше — “большим”.

Это не закон стандарта, а инженерное идиоматическое правило курса для выбора способа передачи объектов в функции.

5.16 С какой целью функция, получающая параметр по ссылке, может возвращать тот же параметр тоже по ссылке (pass-through)?

Это приём pass-through.
Он нужен, чтобы:

- не создавать копию;
- сохранить ссылочную семантику;
- поддержать цепочки вызовов.

Пример

```
int& pass(int& x)
{
    return x;
}
```

6 Стандартная библиотека: основные концепции и линейные контейнеры

6.1 Что такое стандартная библиотека C++ и зачем она нужна?

Стандартная библиотека — часть стандарта C++, предоставляющая готовые типы, алгоритмы, потоки, контейнеры и другие средства общего назначения.

Она нужна, чтобы не реализовывать заново базовые вещи: строки, потоки, контейнеры, итераторы, алгоритмы и т.д.

6.2 Что такое контейнер STL? Какие большие группы контейнеров выделяют?

Контейнер STL — тип данных, предназначенный для хранения коллекции элементов.
В лекциях выделялись три большие группы:

- линейные контейнеры;
- ассоциативные контейнеры;
- неупорядоченные контейнеры.

6.3 Какие контейнеры относятся к линейным?

К линейным контейнерам относились:

- `std::array`;
- `std::vector`;
- `std::deque`;
- `std::forward_list`;
- `std::list`.

6.4 Что такое `std::vector`? В чём его главные свойства?

`std::vector` — линейный контейнер динамического размера.
Главные свойства:

- хранит элементы подряд в памяти;
- даёт быстрый доступ по индексу;
- поддерживает рост контейнера;
- при нехватке памяти может перевыделять внутренний буфер.

6.5 Чем `std::vector` отличается от C-массива и от `std::array`?

Общее с массивами:

- последовательное расположение в памяти;
- быстрый доступ по индексу.

Отличия:

- C-массив не знает своего размера как объектного интерфейса;
- `std::array` имеет фиксированный размер;
- `std::vector` может динамически расти.

6.6 Что такое size и capacity у `std::vector`?

- `size()` — текущее число элементов;
- `capacity()` — объём памяти, уже зарезервированный под элементы без нового перевыделения.

При добавлении элементов capacity растёт не по одному, а обычно скачками.

6.7 Для чего нужен `reserve()`?

`reserve(n)` заранее резервирует память под не менее чем `n` элементов.
Это полезно, чтобы:

- уменьшить число перевыделений памяти;
- не инвалидировать итераторы лишний раз;
- ускорить массовое добавление элементов.

6.8 Что такое матрица в контексте STL-решений курса?

В курсе матрицу удобно представлять как “вектор векторов”:

```
std::vector<std::vector<int>>
```

Это естественное и простое представление двумерной структуры через линейные контейнеры.

6.9 Что такое алиас типа? Чем `using` отличается от `typedef`?

Алиас типа — новое имя для уже существующего типа.
Через `typedef`:

```
typedef std::vector<int> Row;
```

Через `using`:

```
using Row = std::vector<int>;
```

Обе записи делают одно и то же, но `using` обычно считается более современным и удобным.

6.10 Приведите пример алиасов для ряда и зрительного зала.

```
using Row = std::vector<int>;
using SittingPlan = std::vector<Row>;
```

7 Стандартная библиотека: итераторы

7.1 Что такое итератор?

Итератор — объект, который может итерировать элементы некоторой коллекции, то есть перемещаться от элемента к элементу.

Он задаёт определённую позицию в контейнере и служит мостом между контейнерами и алгоритмами.

7.2 Что означают `begin()` и `end()`? Почему интервал записывают как `[begin, end)`?

- `begin()` — итератор на первый элемент;
- `end()` — итератор на позицию *за последним* элементом.

Интервал записывают как `[begin, end)`, то есть левая граница включена, правая не включена.

Поэтому:

- `*begin()` обычно допустимо для непустого контейнера;
- `*end()` — недопустимо.

7.3 Можно ли сделать копию итератора `c.begin()`? Можно ли эту копию разыменовать? Можно ли эту копию двигать вперёд?

Да:

- копию сделать можно;
- разыменовать обычно можно, если контейнер не пуст;
- двигать вперёд можно, если это допускает категория итератора.

7.4 Можно ли сделать копию итератора `c.end()`? Можно ли эту копию разыменовать? Можно ли эту копию двигать назад?

Копию сделать можно.

Разыменовывать `end()` нельзя.

Двигать назад можно не всегда, а только если категория итератора не ниже bidirectional и если это осмысленно для данного контейнера и позиции.

7.5 Какой вариант сравнения итераторов при переборе коллекции следует предпочесть: `it != end()` или `it < end()`? Почему?

Следует предпочитать:

```
it != end()
```

Причина: сравнение `<` поддерживают не все итераторы. Оно доступно только для random access итераторов. А `!=` подходит гораздо более широко.

7.6 Какие категории итераторов выделяли до стандарта C++20? Чем они различаются? Как категории соотносятся друг с другом?

В упрощённом виде в лекции разбирались:

- **forward**;
- **bidirectional**;
- **random access**.

Соотношение такое:

- random access включает возможности bidirectional;
- bidirectional включает возможности forward.

Различаются набором допустимых операций:

- forward: `++it`, `it++`;
- bidirectional: ещё и `-it`, `it-`;
- random access: ещё и `it + n`, `it - n`, сравнение порядка и т.д.

7.7 При разработке нового алгоритма какую категорию итераторов следует требовать?

Правильная политика: требовать **минимальную** категорию итераторов, которая действительно нужна алгоритму.

Это делает алгоритм применимым к большему числу контейнеров.

7.8 Что происходит с итератором после удаления элемента из коллекции? Как следует поступить с этим итератором после удаления элемента?

После удаления элемента итератор на этот элемент обычно становится **недействительным**.

Общая безопасная политика:

- не использовать старый инвалидированный итератор;
- брать новый итератор из возвращаемого значения операции удаления, если контейнер это поддерживает.

8 Косвенная адресация

8.1 Что такое косвенная адресация? Какие основные варианты косвенности разбирались в курсе?

Косвенная адресация — это обращение к объекту не напрямую, а через некоторую “прослойку”.

В курсе разбирались:

- указатели;
- ссылки;
- итераторы;
- `std::string_view`.

8.2 В чём общая идея указателя, ссылки и итератора как средств косвенности?

Во всех трёх случаях мы работаем не с объектом “сам по себе”, а через сущность, которая позволяет к нему обратиться.

Различия:

- указатель требует явного разыменования;
- ссылка ведёт себя почти как само имя объекта;
- итератор ведёт к элементу контейнера и умеет “двигаться” по коллекции.

8.3 Чем указатель отличается от ссылки?

Указатель:

- является отдельным объектом;
- хранит адрес;
- может быть перенаправлен;
- требует разыменования `*` или `->`.

Ссылка:

- обычно рассматривается как ещё одно имя объекта;
- после инициализации не перепривязывается;
- не требует явного разыменования.

8.4 Что значит левоконстантный, правоконстантный и дважды-константный указатель?

- `const int* p;` — константен объект по адресу, но не сам указатель;
- `int* const p = ...;` — константен сам указатель, но объект по адресу можно менять;
- `const int* const p = ...;` — константны и указатель, и объект по адресу.

Именно это в курсе и называли различными разновидностями константности указателя.

8.5 Что можно сказать о константной и неконстантной ссылке?

- `T&` — неконстантная ссылка, через неё можно менять объект;
- `const T&` — константная ссылка, через неё можно только читать объект.

8.6 Что такое `const_iterator`, `const iterator` и `const const_iterator`?

- `const_iterator` — итератор, который не позволяет изменять элемент коллекции;
- `const iterator` — константный объект-итератор, который нельзя "передвигать", но тип элемента может зависеть от самого итератора;
- `const const_iterator` — одновременно константен сам объект-итератор и запрещено менять элемент через него.

Здесь важно различать:

- константность **объекта**;
- константность **косвенности**.

9 Класс std::string_view

9.1 Что такое `std::string_view`?

`std::string_view` — это легковесная невладеющая обёртка над строкой или подстрокой. Он не владеет символами, а только "смотрит" на уже существующий диапазон символов.

9.2 Почему `std::string_view` называют невладеющим типом?

Потому что он не управляет временем жизни символов, на которые указывает.

Следовательно, строка, на которую смотрит `string_view`, должна жить дольше самого `string_view`.

9.3 Почему `std::string_view` удобно передавать по значению?

Потому что это лёгкий тип: обычно он содержит только указатель и длину.

Из-за этого его копирование дешёвое, и правило передачи "малых" объектов по значению здесь подходит естественно.

9.4 Можно ли модифицировать строку через `std::string_view`?

Нет. В лекциях подчёркивалось, что `std::string_view` — **immutable** и допускает только немодифицирующие операции над строкой.

9.5 Гарантирует ли `std::string_view` наличие `'\0'` в конце?

Нет, не гарантирует.

Поэтому его неудобно напрямую использовать с legacy C-кодом, который ожидает классическую C-строку.

9.6 В каком случае `std::string_view` особенно хорош?

Когда нужно эффективно исследовать строки и подстроки без копирования и без модификации исходных данных.

9.7 Как `std::string_view` связан с идеей косвенной адресации?

Он является ещё одним примером косвенности: объект `string_view` сам текст не хранит, а лишь косвенно обращается к уже существующим символам.

10 Ассоциативные контейнеры std::set и std::map

10.1 Что такое `std::set`?

`std::set` — ассоциативный контейнер, который хранит элементы без дубликатов.

Элементы автоматически упорядочиваются согласно критерию сравнения.

10.2 Что такое `std::map`?

`std::map` — ассоциативный контейнер, хранящий пары “ключ–значение”.

Его можно рассматривать как ассоциативный массив.

10.3 Что является элементом `std::map`?

Элемент `std::map` представляется объектом типа `std::pair<Key, Value>`.

10.4 Какой тип обычно используют для представления пары объектов?

Для этого используют служебный тип:

```
std::pair<Type1, Type2>
```

У него есть поля:

- `first`;
- `second`.

10.5 Какая типичная внутренняя реализация у `std::set` и `std::map`?

Типичная реализация — сбалансированное бинарное дерево поиска, обычно красно-чёрное дерево.

10.6 Какова асимптотика основных операций у `std::set` и `std::map`?

Для основных операций:

- поиск — `O(log n)`;
- вставка — `O(log n)`;
- удаление — `O(log n)`.

Полный обход всех элементов — `O(n)`.

10.7 Почему при поиске и вставке в `set/map` используется сравнение?

Потому что контейнеры организованы как дерево поиска, и на каждом шаге нужно решать, идти "влево" или "вправо".

Для этого и нужен критерий "меньше".

10.8 Почему для `std::set` обычно достаточно `operator<`?

Потому что стандартный компаратор по умолчанию — `std::less<Key>`, а он, в простейшем случае, использует именно `operator<`.

10.9 Можно ли хранить в `set` дубликаты?

Нет, `std::set` хранит элементы без дубликатов.

Если нужны повторяющиеся ключи, используют `std::multiset`.

10.10 Можно ли хранить в `map` несколько одинаковых ключей?

Нет, в `std::map` ключи уникальны.

Если нужны повторяющиеся ключи, используют `std::multimap`.

11 Ассоциативные контейнеры, компараторы, сравнение и сортировка

11.1 Каково назначение параметра `Compare` в шаблонах `std::set<Key, Compare>` и `std::map<Key, Value, Compare>`?

Параметр `Compare` задаёт правило упорядочивания элементов внутри контейнера.

Именно через него контейнер понимает, какой из двух ключей "меньше" и как размещать элементы в дереве.

11.2 Что такое компаратор?

Компаратор — объект или функция, задающая отношение порядка между двумя значениями.

В простейшем случае он отвечает на вопрос: должно ли первое значение идти раньше второго.

11.3 Что сравнивает компаратор у `std::set` и у `std::map`?

- у `std::set<Key, Compare>` компаратор сравнивает элементы типа `Key`;
- у `std::map<Key, Value, Compare>` компаратор сравнивает только **ключи** типа `Key`.

11.4 Что такое `std::less`?

`std::less<T>` — стандартный компаратор по умолчанию.

В простейшем варианте он использует `operator<`:

```
lhs < rhs
```

11.5 Можно ли задать свой компаратор?

Да.

Это можно сделать:

- через функциональный объект;
- через тип указателя на функцию;
- через лямбда-выражение, если это поддержано выбранной формой записи.

11.6 Что такое свободная функция в роли компаратора?

Это обычная функция, которую можно использовать для сравнения двух значений.

Например, тип указателя на такую функцию можно описать так:

```
using MyCompareFunPtr = bool(*)(int, int);
```

11.7 Почему компаратор должен задавать устойчивое и разумное правило порядка?

Потому что от него зависит структура дерева, а значит:

- корректность поиска;
- корректность вставки;
- отсутствие логических противоречий в порядке элементов.

Если правило сравнения “ломаное”, контейнер ведёт себя непредсказуемо.

11.8 Какая типичная внутренняя реализация у `std::map` и какова асимптотика операций?

Как и `std::set`, `std::map` обычно реализуется как сбалансированное бинарное дерево поиска, чаще всего красно-чёрное дерево.

Отсюда:

- поиск — `O(log n)`;
- вставка — `O(log n)`;
- удаление — `O(log n)`;
- обход — `O(n)`.

11.9 Можно ли использовать один и тот же компаратор в разных контейнерах?

Да, если этот компаратор умеет сравнивать те значения, которые надо сравнивать в данном контейнере.

Например:

- для `std::set<T>` он сравнивает элементы типа `T`;
- для `std::map<T, U>` — ключи типа `T`;
- для `std::vector<T>` при сортировке — элементы типа `T`.

Главное, чтобы смысл порядка был согласован с задачей.